

Design Document

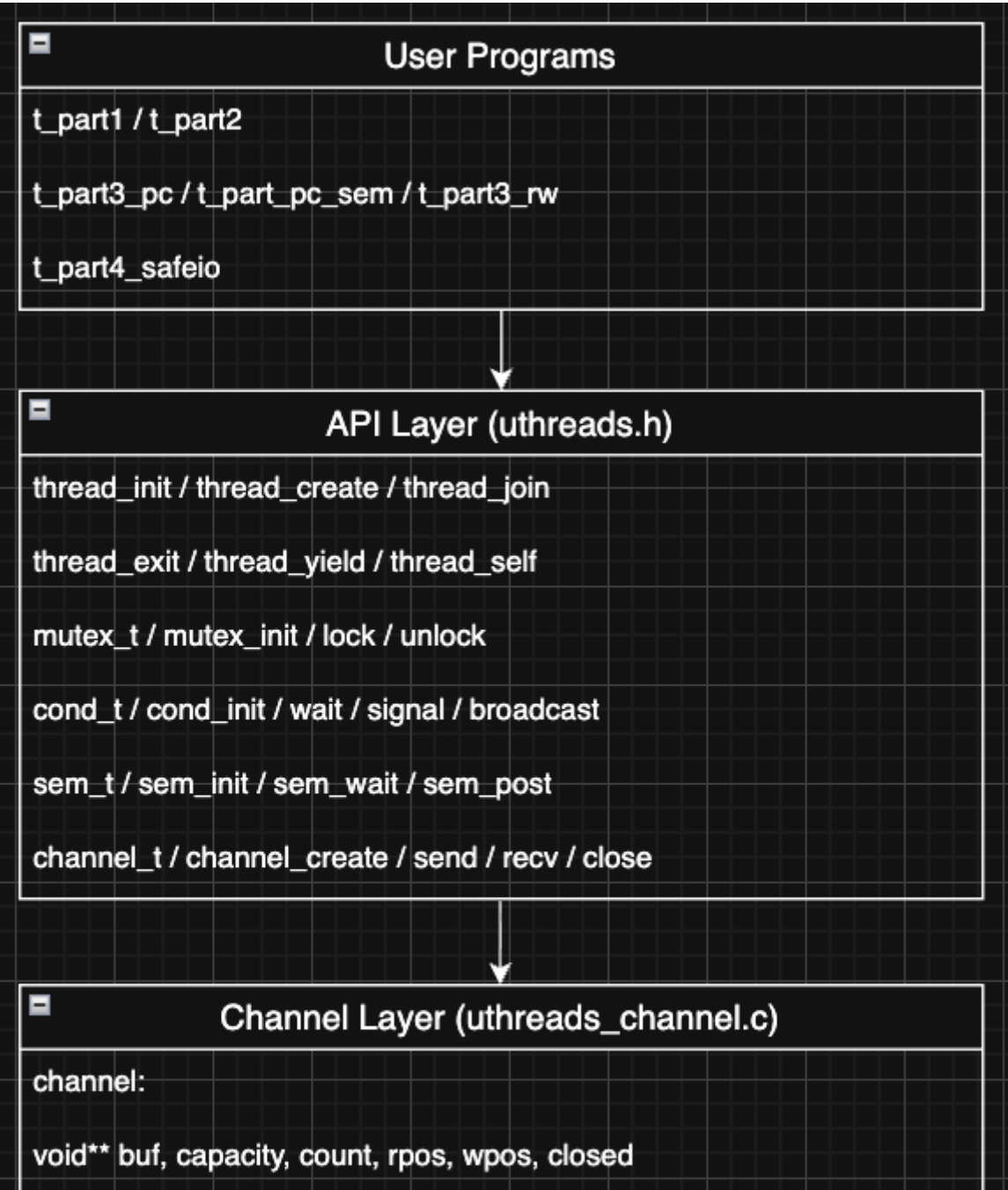
Seongjae Jung (sj4771) & David Hong (sh8348)

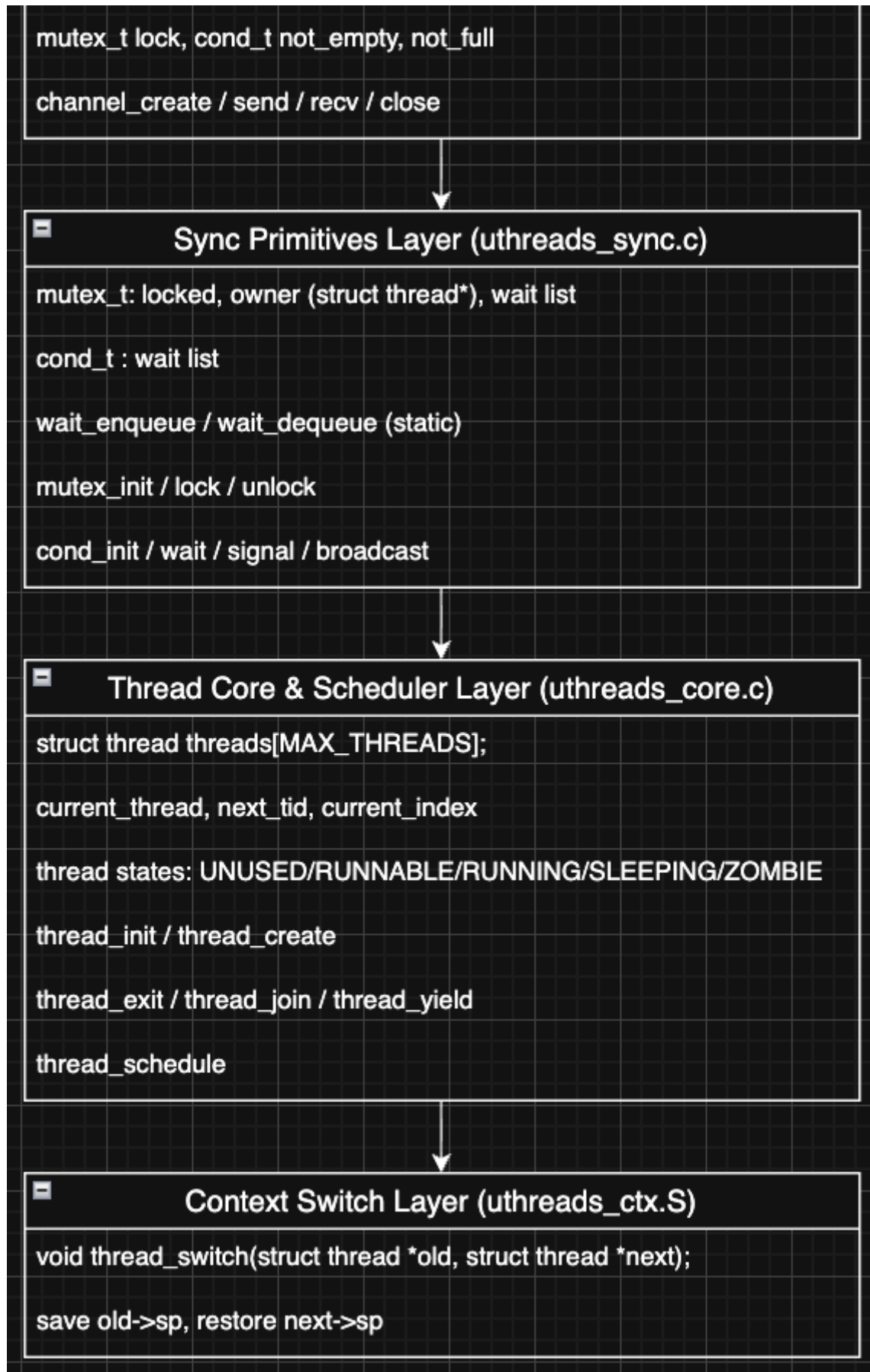
1. Introduction

We built a user-level threading library for xv6. The idea is pretty straightforward — the kernel only sees one process, but inside that process we manage multiple threads ourselves. All the scheduling, context switching, and synchronization happens in user space without the kernel knowing anything about it.

This document goes through our design choices and shows the test results for each part.

2. Architecture Overview





Our library is organized in layers. At the bottom is the assembly code that actually switches between threads. Above that is the scheduler and thread management code in C. Then we have synchronization primitives (mutex, condition variables, semaphores) that build on top of the thread states. Channels sit on top of those, using mutex and condition variables internally. User programs only interact with the top-level API.

We went with this layered approach because it keeps things clean. Each layer only talks to the one below it, so debugging is easier. When something breaks, we can usually figure out which layer is causing the problem.

Files:

- `uthreads.h` — the public API that user programs include
 - `uthreads_private.h` — internal stuff like thread struct and constants
 - `uthreads_core.c` — thread create/join/exit/yield and the scheduler
 - `uthreads_sync.c` — mutex, cond, semaphore implementations
 - `uthreads_channel.c` — bounded buffer channels
 - `uthreads_ctx.S` — the context switch in assembly
 - For extra credit, we also have `safeio.c` which does thread-safe file I/O.
-

3. Thread Structure and States

Each thread is represented by a struct that holds everything we need:

- **sp** — the saved stack pointer, updated during context switch
- **stack** — pointer to the malloc'd stack memory
- **tid** — unique thread ID
- **state** — one of UNUSED, RUNNABLE, RUNNING, SLEEPING, ZOMBIE
- **start_routine, arg** — the function to run and its argument
- **retval** — what the thread returned (picked up by join)
- **joiner** — points to whoever is waiting in `thread_join` for us
- **wait_next** — used to link threads in wait queues

We picked `MAX_THREADS = 64` and `STACK_SIZE = 8KB`. 64 threads seemed like plenty for any reasonable test, and 8KB is enough stack space for typical function calls without wasting too much memory.

State transitions:

- UNUSED → RUNNABLE: when `thread_create` allocates a slot
 - RUNNABLE → RUNNING: scheduler picks the thread
 - RUNNING → RUNNABLE: thread yields or gets switched out
 - RUNNING → SLEEPING: thread blocks on mutex/cond/semaphore/join
 - SLEEPING → RUNNABLE: another thread wakes us up
 - RUNNING → ZOMBIE: `thread_exit` is called
 - ZOMBIE → UNUSED: join collects the return value and cleans up
-

4. Context Switching

This is the tricky part. We can't do this in C because C doesn't let you mess with the stack pointer directly. So we wrote it in x86 assembly.

The idea: save the current thread's registers onto its stack, save the stack pointer into the thread struct, load the new thread's stack pointer, pop its registers, and return. The "return" actually jumps to wherever the new thread was when it last switched out.

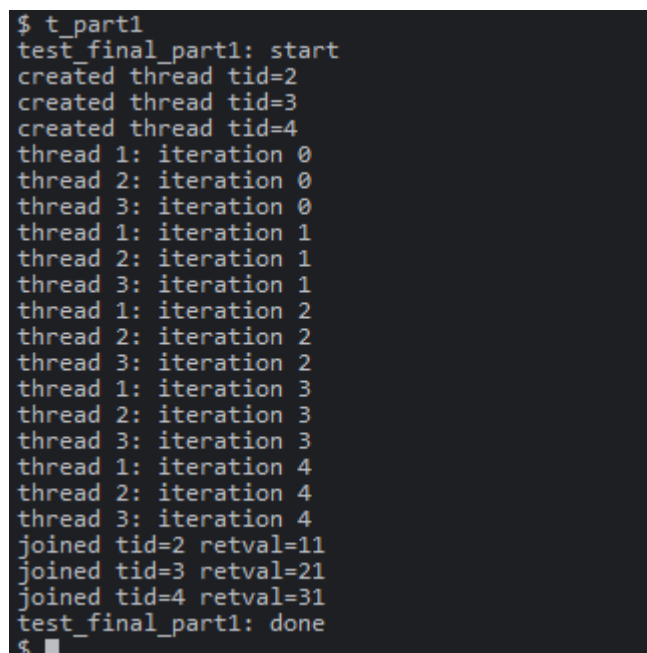
For brand new threads, we set up the stack so that when we "return" into them for the first time, we land in a trampoline function. The trampoline calls the user's start_routine, and when that returns, it calls thread_exit with the return value.

We only save the callee-saved registers (ebx, esi, edi, ebp). The calling convention says the caller already assumes eax/ecx/edx might be trashed, so we don't bother saving those.

5. Scheduler

We kept the scheduler simple — just round-robin. Starting from the current thread's position in the table, we scan forward looking for the next RUNNABLE thread. If we find one, we switch to it. If nothing else is runnable, we just keep running.

6. Part 1: Threading Foundation



```
$ t_part1
test_final_part1: start
created thread tid=2
created thread tid=3
created thread tid=4
thread 1: iteration 0
thread 2: iteration 0
thread 3: iteration 0
thread 1: iteration 1
thread 2: iteration 1
thread 3: iteration 1
thread 1: iteration 2
thread 2: iteration 2
thread 3: iteration 2
thread 1: iteration 3
thread 2: iteration 3
thread 3: iteration 3
thread 1: iteration 4
thread 2: iteration 4
thread 3: iteration 4
joined tid=2 retval=11
joined tid=3 retval=21
joined tid=4 retval=31
test_final_part1: done
$
```

This tests the basics — can we create threads, run them, and join them?

We spawn 3 threads. Each one prints 5 iterations, yielding between each print. At the end, main joins all three and collects their return values.

What's happening in the output:

- Threads are created with tid 2, 3, 4 (main is tid 1)
- They interleave nicely — thread 1 prints iteration 0, then thread 2, then thread 3, etc.
- Each thread returns (tid * 10 + 1), so we see retval=11, 21, 31

This shows that thread creation, yielding, scheduling, and join all work correctly.

7. Part 2: Synchronization Primitives

```
$ t_part2 _
test_final_part2: start

[Shared Counter NO mutex]
[no_lock] thread 1: i=0
[no_lock] thread 2: i=0
[no_lock] thread 3: i=0
[no_lock] thread 4: i=0
[no_lock] thread 1: i=200
[no_lock] thread 2: i=200
[no_lock] thread 3: i=200
[no_lock] thread 4: i=200
[no_lock] thread 1: i=400
[no_lock] thread 2: i=400
[no_lock] thread 3: i=400
[no_lock] thread 4: i=400
[no_lock] thread 1: i=600
[no_lock] thread 2: i=600
[no_lock] thread 3: i=600
[no_lock] thread 4: i=600
[no_lock] thread 1: i=800
[no_lock] thread 2: i=800
[no_lock] thread 3: i=800
[no_lock] thread 4: i=800
counter_no_lock final value = 4000 (expected 4000)

[Shared Counter WITH mutex]
[mutex] thread 1: i=0
[mutex] thread 2: i=0
[mutex] thread 3: i=0
[mutex] thread 4: i=0
[mutex] thread 1: i=200
[mutex] thread 2: i=200
[mutex] thread 3: i=200
[mutex] thread 4: i=200
[mutex] thread 1: i=400
[mutex] thread 2: i=400
[mutex] thread 3: i=400
[mutex] thread 4: i=400
[mutex] thread 1: i=600
[mutex] thread 2: i=600
[mutex] thread 3: i=600
[mutex] thread 4: i=600
[mutex] thread 1: i=800
[mutex] thread 2: i=800
[mutex] thread 3: i=800
[mutex] thread 4: i=800
counter final value = 4000 (expected 4000)

[Condition Variable Test]
cond_waiter: waiting
cond_signaler: work 0
cond_signaler: work 1
cond_signaler: work 2
cond_signaler: work 3
cond_signaler: work 4
cond_waiter: got signal

[Channel Test]
channel_producer: start
channel_producer: sent 0
channel_consumer: start
channel_consumer: got 0
channel_consumer: got 1
channel_consumer: got 2
channel_consumer: got 3
channel_consumer: got 4
channel_producer: sent 5
channel_consumer: got 5
channel_consumer: got 6
channel_consumer: got 7
channel_consumer: got 8
channel_consumer: got 9
channel_producer: sent 10
channel_consumer: got 10
channel_consumer: got 11
```

```
channel_consumer: got 12
channel_consumer: got 13
channel_consumer: got 14
channel_producer: sent 15
channel_consumer: got 15
channel_producer: done
channel_consumer: got 16
channel_consumer: got 17
channel_consumer: got 18
channel_consumer: got 19
channel_consumer: received 20 items
channel_test: done
test_final_part2: done
```

This test covers mutex, shared counter, semaphores, condition variables, and channels.

Mutex

We have a mutex that tracks: is it locked, who owns it, and a queue of waiting threads.

mutex_lock: If it's free, grab it. If not, add ourselves to the wait queue, mark ourselves SLEEPING, and call the scheduler.

mutex_unlock: If someone's waiting, hand ownership directly to them and wake them up. Otherwise just mark it unlocked.

Shared Counter: Race Condition Demo

To show why mutexes matter, we run the same test twice — once without any locking (`[no_lock]`), once with the mutex (`[mutex]`).

The test spawns 4 threads, each incrementing a shared counter 1000 times. Expected result: $4 \times 1000 = 4000$.

Without mutex:

Each thread does `counter++`, which looks like one operation but is actually three: load the value, add 1, store it back. In theory, if thread A loads 100, then thread B loads 100, both add 1 and store 101 — we lose an increment.

In the screenshot, both versions happen to show 4000. This is because our library uses cooperative scheduling — threads only switch when they explicitly call `thread_yield()`. Since the increment happens between yields, no interleaving occurs at that point. In a preemptive system, you'd see lost updates.

With mutex:

Same code, but we wrap the increment in `mutex_lock/mutex_unlock`. This guarantees correctness regardless of scheduling — only one thread can touch the counter at a time.

The screenshot shows `counter final value = 4000 (expected 4000)`. The mutex version is always correct, even if preemption were added later.

Semaphores

Semaphores are like mutexes but with a counter instead of just locked/unlocked. We track a value and a wait queue.

sem_wait: Decrement the value. If it goes negative, we need to block — add ourselves to the wait queue, set state to SLEEPING, and call the scheduler.

sem_post: Increment the value. If it was negative (meaning someone was waiting), wake up one thread from the queue.

The nice thing about semaphores is they can handle more than just mutual exclusion. You can use them for signaling between threads or counting available resources. We use them in the producer-consumer problem to track empty slots and filled slots in the buffer.

Condition Variables

A condition variable is basically a wait queue. `cond_wait` adds you to the queue, releases the mutex (so others can make progress), goes to sleep, and re-acquires the mutex when woken up. `cond_signal` wakes one waiter, `cond_broadcast` wakes everyone.

The test has a waiter thread and a signaler thread. The waiter sits in `cond_wait` until the signaler finishes its work and sends a signal.

Channels

Channels are bounded buffers. We have a mutex protecting the buffer state, plus two condition variables — one for "buffer not empty" and one for "buffer not full". `Send` blocks if full, `recv` blocks if empty. `Close` wakes everyone up so they can exit.

The channel test shows a producer sending items 0-19 and a consumer receiving them. "received 20 items" confirms nothing was lost.

8. Part 3: Producer-Consumer with Semaphores

```
$ t_part3_pc_sem
test_final_part3_producer_consumer_sem: start
[SEM] Consumer 1: start
[SEM] Consumer 2: start
[SEM] Producer 1: start
[SEM] Producer 1: produced item 0 (val=100)
[SEM] Producer 2: start
[SEM] Producer 2: produced item 0 (val=200)
[SEM] Producer 3: start
[SEM] Producer 3: produced item 0 (val=300)
[SEM] Consumer 1: consumed val=100
[SEM] Consumer 1: consumed val=200
[SEM] Consumer 2: consumed val=300
[SEM] Producer 1: produced item 1 (val=101)
[SEM] Producer 1: produced item 2 (val=102)
[SEM] Producer 1: produced item 3 (val=103)
[SEM] Producer 2: produced item 1 (val=201)
[SEM] Producer 2: produced item 2 (val=202)
[SEM] Consumer 1: consumed val=101
[SEM] Consumer 1: consumed val=102
[SEM] Consumer 2: consumed val=103
[SEM] Consumer 2: consumed val=201
[SEM] Consumer 2: consumed val=202
[SEM] Producer 1: produced item 4 (val=104)
[SEM] Producer 1: produced item 5 (val=105)
[SEM] Producer 1: produced item 6 (val=106)
[SEM] Producer 2: produced item 3 (val=203)
[SEM] Producer 3: produced item 1 (val=301)
[SEM] Consumer 1: consumed val=104
[SEM] Consumer 1: consumed val=105
[SEM] Consumer 1: consumed val=106
[SEM] Consumer 1: consumed val=203
```

```

[SEM] Consumer 2: consumed val=301
[SEM] Producer 1: produced item 7 (val=107)
[SEM] Producer 1: produced item 8 (val=108)
[SEM] Producer 1: produced item 9 (val=109)
[SEM] Producer 2: produced item 4 (val=204)
[SEM] Producer 3: produced item 2 (val=302)
[SEM] Consumer 1: consumed val=107
[SEM] Consumer 1: consumed val=108
[SEM] Consumer 1: consumed val=109
[SEM] Consumer 1: consumed val=204
[SEM] Consumer 2: consumed val=302
[SEM] Producer 1: done
[SEM] Producer 2: produced item 5 (val=205)
[SEM] Producer 2: produced item 6 (val=206)
[SEM] Producer 3: produced item 3 (val=303)
main: joined producer tid=4
[SEM] Consumer 1: consumed val=205
[SEM] Consumer 1: consumed val=206
[SEM] Consumer 2: consumed val=303
[SEM] Producer 2: produced item 7 (val=207)
[SEM] Producer 2: produced item 8 (val=208)
[SEM] Producer 2: produced item 9 (val=209)
[SEM] Producer 3: produced item 4 (val=304)
[SEM] Producer 3: produced item 5 (val=305)
[SEM] Consumer 1: consumed val=207
[SEM] Consumer 1: consumed val=208
[SEM] Consumer 2: consumed val=209
[SEM] Producer 2: done
[SEM] Producer 3: produced item 6 (val=306)
main: joined producer tid=5
[SEM] Consumer 1: consumed val=304
[SEM] Consumer 1: consumed val=305
[SEM] Consumer 1: consumed val=306
[SEM] Producer 3: produced item 7 (val=307)
[SEM] Producer 3: produced item 8 (val=308)
[SEM] Producer 3: produced item 9 (val=309)
[SEM] Consumer 1: consumed val=307
[SEM] Consumer 2: consumed val=308
[SEM] Consumer 2: consumed val=309
[SEM] Producer 3: done
main: joined producer tid=6
[SEM] Consumer 1: got sentinel, exit
[SEM] Consumer 1: consumed 20 real items
[SEM] Consumer 2: got sentinel, exit
[SEM] Consumer 2: consumed 10 real items
main: joined consumer tid=2
main: joined consumer tid=3
produced_items = 30 (expected 30)
consumed_items = 30 (expected 30)
test_final_part3_producer_consumer_sem: done

```

This is the classic producer-consumer problem solved using semaphores. 3 producers, 2 consumers, buffer size 5.

We use three synchronization primitives:

- `empty_slots` semaphore (init = 5) — tracks free space in buffer
- `full_slots` semaphore (init = 0) — tracks items waiting to be consumed
- `buf_lock` mutex — protects the actual buffer array

Producer flow:

1. `sem_wait(empty_slots)` — wait for a free slot
2. `mutex_lock(buf_lock)` — grab exclusive access to buffer
3. write item to buffer
4. `mutex_unlock(buf_lock)`
5. `sem_post(full_slots)` — signal that there's a new item

Consumer flow:

1. `sem_wait(full_slots)` — wait for an item
2. `mutex_lock(buf_lock)` — grab exclusive access
3. read item from buffer
4. `mutex_unlock(buf_lock)`
5. `sem_post(empty_slots)` — signal that there's a free slot

In the output, you can see `[SEM]` prefix on all the log messages. Producers generate items (100-109, 200-209, 300-309) and consumers grab them. Consumer 1 consumed 20 items, Consumer 2 consumed 10 items. At the end: `produced_items = 30`, `consumed_items = 30` — everything checks out.

To shut down cleanly, after all producers finish, main sends sentinel values (-1) to tell consumers to exit.

9. Part 3: Producer-Consumer with Channels

```
$ t_part3_pc
test_final_part3_producer_consumer: start
Producer 1: start
Producer 1: generated item 0 (val=100)
Producer 2: start
Producer 2: generated item 0 (val=200)
Producer 3: start
Producer 3: generated item 0 (val=300)
Consumer 1: start
Consumer 1: consumed val=100
Consumer 1: consumed val=200
Consumer 1: consumed val=300
Consumer 2: start
Producer 1: generated item 1 (val=101)
Producer 1: generated item 2 (val=102)
Producer 1: generated item 3 (val=103)
Producer 2: generated item 1 (val=201)
Producer 2: generated item 2 (val=202)
Producer 2: generated item 3 (val=203)
Producer 3: generated item 1 (val=301)
Producer 3: generated item 2 (val=302)
Consumer 1: consumed val=101
Consumer 2: consumed val=102
Consumer 2: consumed val=103
Consumer 2: consumed val=201
Consumer 2: consumed val=202
Producer 1: generated item 4 (val=104)
Producer 1: generated item 5 (val=105)
Producer 1: generated item 6 (val=106)
Producer 2: generated item 4 (val=204)
Producer 2: generated item 5 (val=205)
Consumer 1: consumed val=203
Consumer 1: consumed val=301
Consumer 1: consumed val=302
Consumer 1: consumed val=104
Consumer 2: consumed val=105
Consumer 2: consumed val=106
Consumer 2: consumed val=204
Consumer 2: consumed val=205
Producer 1: generated item 7 (val=107)
Producer 1: generated item 8 (val=108)
Producer 1: generated item 9 (val=109)
Producer 2: generated item 6 (val=206)
Producer 3: generated item 3 (val=303)
Consumer 1: consumed val=107
Consumer 1: consumed val=108
Consumer 1: consumed val=109
Consumer 1: consumed val=206
Consumer 2: consumed val=303
Producer 1: done
Producer 2: generated item 7 (val=207)
Producer 2: generated item 8 (val=208)
Producer 2: generated item 9 (val=209)
Producer 3: generated item 4 (val=304)
Producer 3: generated item 5 (val=305)
Producer 3: generated item 6 (val=306)
Consumer 1: consumed val=207
Consumer 1: consumed val=208
Consumer 1: consumed val=209
Consumer 1: consumed val=304
Consumer 2: consumed val=305
Consumer 2: consumed val=306
main: joined producer tid=2
Producer 2: done
Producer 3: generated item 7 (val=307)
Producer 3: generated item 8 (val=308)
Producer 3: generated item 9 (val=309)
Consumer 1: consumed val=307
Consumer 1: consumed val=308
Consumer 1: consumed val=309
main: joined producer tid=3
Producer 3: done
main: joined producer tid=4
Consumer 1: received 19 items, exit
Consumer 2: received 11 items, exit
main: joined consumer tid=5
main: joined consumer tid=6
test_final_part3_producer_consumer: done
```

Same problem as above, but using channels instead. Much cleaner code.

Each producer generates 10 items. The value encodes which producer made it (producer 1 makes 100-109, producer 2 makes 200-209, etc.). Consumers just grab whatever's available and print it.

Looking at the output:

- Producers and consumers interleave naturally
- All 30 items get produced and consumed
- Consumer 1 got 19 items, Consumer 2 got 11 items
- After `channel_close()`, the `recv` calls return -1 and consumers exit

The channel handles all the synchronization internally. No explicit mutex or semaphore calls in the producer/consumer code — just send and recv. This is why channels are nice to work with.

10. Part 3: Reader-Writer Lock

```
$ t_part3_rw
test_final_part3_reader_writer: start
Reader 1: start
Reader 1: value = 0
Reader 2: start
Reader 2: value = 0
Reader 3: start
Reader 3: value = 0
Writer 1: start
Writer 1: wrote value = 1
Writer 2: start
Writer 2: wrote value = 2
Reader 1: value = 2
Reader 2: value = 2
Reader 3: value = 2
Writer 1: wrote value = 3
Writer 2: wrote value = 4
Reader 1: value = 4
Reader 2: value = 4
Reader 3: value = 4
Writer 1: wrote value = 5
Writer 2: wrote value = 6
Reader 1: value = 6
Reader 2: value = 6
Reader 3: value = 6
Writer 1: done
Writer 2: done
Reader 1: value = 6
Reader 2: value = 6
Reader 3: value = 6
Reader 1: done
Reader 2: done
Reader 3: done
main: joined reader tid=2
main: joined reader tid=3
main: joined reader tid=4
main: joined writer tid=5
main: joined writer tid=6
final_shared_value = 6
test_final_part3_reader_writer: done
```

The reader-writer problem: multiple readers can access data simultaneously, but writers need exclusive access.

We implemented writer priority. If a writer is waiting, new readers won't start — they'll wait until the writer is done. This prevents writer starvation.

State variables:

- **readers** — count of active readers
- **writers_waiting** — count of writers in queue
- **writer_active** — 1 if a writer is currently writing

reader_lock: Wait while (writer_active OR writers_waiting > 0), then increment readers.

writer_lock: Increment writers_waiting, wait while (writer_active OR readers > 0), then decrement writers_waiting and set writer_active.

writer_unlock: Clear writer_active. If more writers waiting, wake one. Otherwise wake all readers.

In the output:

- 3 readers start reading value 0 (they can run concurrently)
- Writer 1 writes value 1, then Writer 2 writes value 2
- Writers alternate with readers, but readers defer when writers are waiting
- Final shared_value = 6 (2 writers × 3 writes each)

11. Part 4: Thread-Safe File I/O

```
$ t_part4_safeio  
test final_extra: start  
reader: start  
reader: waiting for file  
writer: start  
writer: wrote 'hello world!'  
reader: read 'hello world!'  
writer: wrote 'hello world!'  
reader: read 'hello world!'  
writer: wrote 'hello world!'  
reader: read 'hello world!'  
writer: wrote 'hello world!'  
reader: read 'hello world!'  
writer: wrote 'hello world!'  
reader: read 'hello world!'  
writer: wrote 'hello world!'  
reader: read 'hello world!'  
writer: wrote 'hello world!'  
reader: read 'hello world!'  
writer: wrote 'hello world!'  
reader: read 'hello world!'  
writer: wrote 'hello world!'  
reader: read 'hello world!'  
writer: wrote 'hello world!'  
reader: read 'hello world!'  
writer: wrote 'hello world!'  
reader: read 'hello world!'  
writer: wrote 'hello world!'  
reader: read 'hello world!'  
writer: wrote 'hello world!'  
reader: read 'hello world!'  
writer: wrote 'hello world!'  
reader: read 'hello world!'  
writer: wrote 'hello world!'  
reader: read 'hello world!'  
writer: wrote 'hello world!'  
reader: read 'hello world!'  
writer: done  
reader: done  
test final_extra: done
```

The problem with xv6 system calls is they block the whole process. If one thread calls `read()` and blocks, every thread freezes. That's bad for a threading library.

Our solution: a dedicated I/O worker thread.

When you call `safe_read()` or `safe_write()`, we package up the request and send it through a channel to the worker. Then we wait on a condition variable. The worker thread does the actual syscall, stores the result, and signals us. Meanwhile, other threads can keep running.

The demo forks into two processes:

- Writer process: opens a file, writes "hello world!" 20 times with sleep(10) between each
- Reader process: waits for the file to appear, then reads and prints whatever shows up

In the output, you see writer and reader interleaving. The reader catches up with the writer, and eventually both finish. This shows that even though `read()` and `write()` are blocking syscalls, our threading system stays responsive.